Comprehensive Reference Summary: SQLPathFinder → DataSyncX Translation Pipeline

Self-reference document compiled after reading all provided SQLPathFinder and DataSyncX documentation. This will ground all later design decisions when actual VG2 scripts are provided.

Section 1: Problem Framing (The Big Picture)

What we're building: An automated pipeline that takes a VG2 file (the XML-like script SQLPathFinder produces) and emits an equivalent Python script that uses the DataSyncX library to run the same data extraction + transformation flow.

Why it's non-trivial (key insight from datasyncx_reader_fact.txt):

- VG2 carries **physical table names + per-column expressions**, but SPF *logical views* (e.g., F_Lot_History_v3, P_SPC_Chart_or_Raw_Data) are **not real Oracle views**. SPF expands them client-side into FROM clauses + intra-view JOINs + implicit WHEREs at runtime. [datasyncx ...eader_fact | Txt]
- The VG2 **does NOT carry**: intra-view join keys, implicit WHERE conditions, or "bridge" sub-aliases that SPF silently injects (e.g., v2 may be needed to bridge v1↔v3 even though it never appears in the user's selected columns). [datasyncx ...eader_fact | Txt]
- **Implication:** The translator cannot be a pure VG2-parser → Python emitter. It needs a hand-maintained **view-expansion registry** (likely YAML) for every SPF logical view the team actually uses.

Section 2: SQLPathFinder Concept Stack

2.1 What SPF Is

SQLPathFinder (SPF) is Intel's ad-hoc query designer that targets Intel manufacturing engineering DBs (MARS, ARIES/XEUS, OASys, plus others like TCA, SPEED, AEPT). It produces a **VG2 script** (the design artifact) and a **runtime SQL/extract script** that the SPF Extract Engine executes. Users design via a GUI, then either run interactively or schedule on **ScriptHost**. [Introducti...AT_MfgData | PowerPoint] [Introducti...aced_Intro | PowerPoint]

2.2 The Database Landscape (from Introduction_to_SQLPathFinder_SelfPaced_AT_MfgData.ppsm)

Engineering DB	Source System	Key SPF Logical Views	What's Inside
MARS	WorkStream / MES300	WIP_Lot_Status_v2, WIP_Lot_History_v2, Entity_History	Lot status, lot/operation/entity history, PM/repair history
ARIES (XEUS for 1272+ Fabs)	Sort/Class/E-Test testers	Sort_or_Test_Lot_Rollup, Sort_or_Test_Wafer_or_Session_Rollup, Sort_or_Test_Dice_or_Unit_Results, AT_Metrology, ULT_Data	Unit/wafer/lot test bins, parametric, E-Test
OASys	SPC++	SPC_Chart_or_Raw, AT_RFC_Master	SPC raw + chart summary, RFC checklists
Other	AEPT, SPEED, TCA, ULT, etc.	AEPT_Alarm_Data, Speed_AT_Status	Equipment alarms, MRB/DRB, traceability

Logical view **prefix convention** (per datasyncx_reader_fact.txt): F_* = MARS, P_* = OASYS. [datasyncx_...eader_fact | Txt]

2.3 Anatomy of a VG2 Query (synthesized from Creating Queries + Adding Utilities)

A VG2 file is a **tree** rooted at PathFinder with the following top-level node types (from Creating Queries - SQLPathFinder - Intel Enterprise Wiki.pdf and Adding Utilities - SQLPathFinder - Intel Enterprise Wiki.pdf): [Creating Q...prise Wiki | PDF], [Adding Uti...prise Wiki | PDF]

```
PathFinder
├── 1. Main Query
│   ├── Output Options (Distinct Rows, Row Limit, Output File,
│   │   Outer Join, Interleave Results, Pivot Missing After
Join,
│   │   Quote Embedded Commas, Set Empty Strings to Null, Show
SQL)
│   └── 1.1 Views
│       └── <ViewName> a0   (alias = a0, a1, a2…)
│           ├── Columns grid     ← Header, Show, Sort,
Statistic, Pivot, Level
│           ├── Filters grid     ← And/Or, Column, Operator,
Value, Key
│           └── Joins grid       ← inter-view joins
```

```
11  ├── 2. <Utility steps>          ← Rows-In-File, If-Then,
Email, Copy, …
12  └── n. End-If / End-Loop / End-Macro / End-Block
```

2.4 Column Grid Attributes (the meat of "what to SELECT")

Attribute	Meaning / Translation Implication
Column	The source column expression (e.g., a0.etest_lot, a0->p.prodgroup3) — includes sub-alias chains
Header	The output column name (renamed in DataFrame)
Show	Y = include in output, N = drop, P = pass-through across levels, Y:n = significant figures
Sort	ASC / DESC / None; top-down order = multi-key sort
Statistic	AVG, SUM, COUNT, MAX, MIN, STDDEV, VARIANCE, P1..P99 — **non-statistic + Show=Y columns become GROUP BY keys**
Pivot	Row / Header / Value — for pivot queries (rows→columns)
Level	View,0 (per view), Join,0 (all-sources), higher levels build on prior

2.5 Filter Grid (the "WHERE" clause)

Operators (from Introduction_to_SQLPathFinder_SelfPaced_Filtering.ppsm): [Introducti... Filtering | PowerPoint]

- **Scalar:** =, IN, LIKE, LIKE LIST, BETWEEN, >=, >, <=, <
- **Wildcards:** % (0+ chars), _ (single char)
- **File-based:** IN GROUP (search column-1 in a CSV), LIKE GROUP (CSV with wildcards), In Temp (batched IN — used to chunk large lists)
- **AND/OR + parentheses:** AND binds tighter than OR unless parenthesized
- **Keys:** Gold key = first column in DB index (fast); Silver = subsequent. **SPF requires at least one gold-key filter** for most non-status views

2.6 Computed Columns (from Introduction_to_SQLPathFinder_SelfPaced_Computed_Columns_2.ppsm) [Introducti... Columns_2 | PowerPoint]

User-defined expressions added at a chosen **Level**. Three families to translate:

Family	Examples	Notes
Scalar	|| / + concat, SUBSTR, LENGTH, REPLACE, spf_FN$Instr, spf_FN$IFNULL, spf_FN$Date_Diff	The spf_FN$* prefix denotes cross-database normalized functions — they must map to pandas/SQL equivalents
Conditional	CASE WHEN … THEN … ELSE … END (up to 128 WHEN)	Maps cleanly to numpy.select / np.where
Analytical	DENSE_RANK() OVER (PARTITION BY … ORDER BY …), LEAD, LAG, windowed AVG/SUM/STDDEV/percentile	Some are Oracle-only; in pandas use groupby().rank(), shift(), transform()

⚠️ **SQLite gotcha (used at Join/All-Sources level):** integer arithmetic truncates (5/2 = 2). Fixes: multiply by 1.0 or CAST(... AS REAL). This affects how we re-emit cross-source joins in pandas (which doesn't truncate, so we may actually be safer).

2.7 Summaries (from Introduction_to_SQLPathFinder_SelfPaced_Summary.ppsm) [Introducti...ed_Summary | PowerPoint]

- Apply a Statistic in the Columns grid → that column is aggregated.
- All other Show=Y + no-statistic columns become the GROUP BY.
- All aggregates **ignore NULL** except COUNT(*). Use spf_FN$IFNULL(col, 0) to force inclusion.
- All-Stat column = same idea but at the All-Sources (post-join) level.

2.8 Pivots (from Introduction_to_SQLPathFinder_SelfPaced_Pivoting.ppsm) [Introducti...d_Pivoting | PowerPoint]

Columns are tagged in the Pivot column:

- Row columns → the pivot index (kept as-is)
- Header column → its values become the new column headers
- Value columns → cells being transposed; **must also have a Statistic** (e.g., AVG) to guarantee uniqueness

Translation target: pandas.pivot_table(index=[Row…], columns=Header, values=Value, aggfunc=Statistic).

2.9 Global Variables (from Global Variables - SQLPathFinder - Intel Enterprise Wiki.pdf) [Global Var...prise Wiki | PDF]

Three reference syntaxes the VG2 parser must handle:

Syntax	Meaning	Translation strategy
<<<myVar>>>	User-defined global	Resolve at parse time from VG2's global-vars block → Python constant or env lookup
<<<%ENVVAR%>>>	OS environment variable	os.environ['ENVVAR']
<<<spf-…>>> / <<<spf$…>>>	System-defined (e.g., spf-default-dir, spf$facility, spf$node, spf-loop-ctr-1)	Map to runtime context (work folder, site name, loop counter)
<//q//>	Two single-quotes ('')	String-replace pre-emit
/CL_lot="ABC"	Command-line override	Becomes a CLI argument / function parameter in emitted Python

2.10 Utilities Catalog (from Adding Utilities - SQLPathFinder - Intel Enterprise Wiki.pdf) [Adding Uti...prise Wiki | PDF]

Grouped by translation priority:

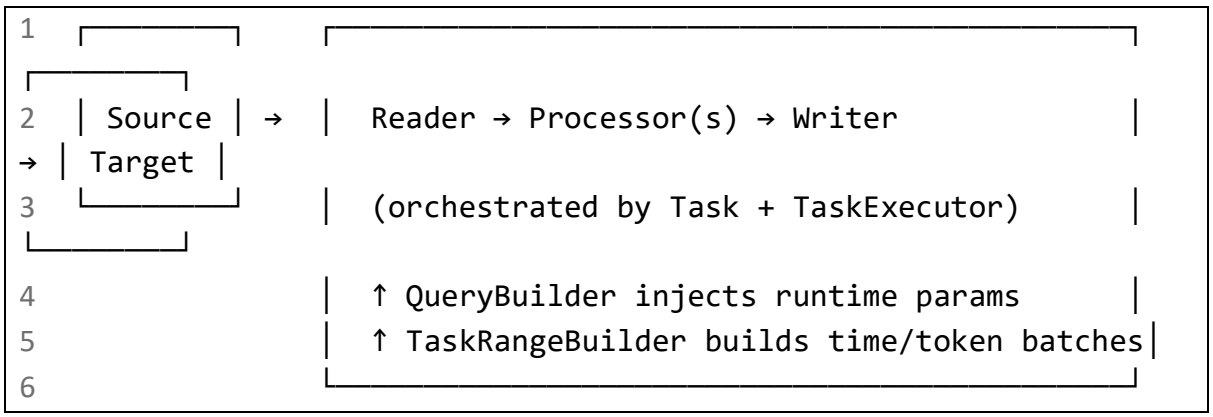
Group	Utilities	Translation Target
File ops	Copy, Rename-Move, Delete, Append,Web-Copy	shutil, pathlib, requests
Logic flow	If-Then-Else (EQ/NE/GT/LT/GE/LE + their S string variants, AND/OR conditions), Rows-In-File, Value-In-File, Age-Of-File, Date-Of-File	Plain Python if/elif/else, pathlib.stat()
Iteration	For-Loop (with <<<spf-loop-ctr-i>>>, <<<spf-start-i>>>, <<<spf-end-i>>>, <<<spf-step-i>>> + DB-specific date macros), Run-Loop, Site-Loop, Begin-Block/End-Block, Start-Macro/End-Macro	for loops; Site-Loop → loop over site list using DataSyncX TaskRangeBuilder per site
Incremental ETL	Smart-Append (v1/v2/v3 — append + update-by-key + delete-by-date), Get-Site-Time, Update-Time	This maps directly to DataSyncX's incremental + watermark patterns. **Smart-Append v3 ≈ DataSyncX merge writer.**
Data export	Email, SharePoint, Excel, JMP, HTML-Report, Chart-in-JMP, Write-File, Write-File-Immediate	Email = smtplib or DataSyncX notification utils; Excel = openpyxl; JMP/HTML usually out-of-scope for v1 translator
Special	Get-TCA (XML→CSV via TCA webservice), TDX-TO-CSV, Mongo-Import, File-Compare, IDEAL-Table-Config, Screen-and-Visualize-Features, Python, R, Pre/Post Query	Most are domain-specific; flag for manual port
HPC / Misc	Begin-HPC/End-HPC, Wait-File, DOS, Prompt-Input	Map to subprocess / time.sleep / argparse

Section 3: DataSyncX Concept Stack

3.1 What It Is (from Concepts — DataSyncX latest documentation.pdf + FAQ — DataSyncX latest documentation.pdf) [Concepts —...umentation | PDF], [FAQ — Data...umentation | PDF]

DataSyncX is a Python ETL framework that enforces a **Reader → Processor → Writer** pipeline with orchestration utilities for query parameterization and task ranges. It's the runtime our generated Python scripts should target.

3.2 Architecture



3.3 Readers (verified facts from datasyncx_reader_fact.txt — **trust this over docs when in doubt**) [datasyncx_...eader_fact | Txt]

Exposed readers: AriesReader, FileReader, ImBigDataReader, LamasReader, LargeFileReader, MongoDBReader/MongoReader, MarsReader, OracleReader, PostgreSQLReader, SapHanaReader, SharePointReader, SQLServerReader/SqlServerReader, TeradataReader, XeusReader.

⚠️ **There is NO**OasysReader. For OASYS, use OracleReader(database='OASYS').

Critical reader behaviors:

Reader	Signature	Schema substitution?
MarsReader(*, username, password, db_account=False)	.read(site, query, factory=...) calls query_mao(...) with database='MARS' hardcoded	**YES** — substitutes :schema and @[]@ with hardcoded schema map (e.g., KM → A15_PROD_21)
OracleReader(*, sites, username, password, database, node)	.read(site, query) calls query_oracle(...) with dsn=f'{site}.{database}'	**NO** — pass full SQL with explicit schema

Hardcoded MARS schema map (from utils/oracle_db.py): CDDP=A46_PROD_11, AFODP=AAL_PROD_1, KMDP=A15_PROD_21, AFO=AAL_PROD_1, CD=A48_PROD_21, PG=A12_PROD_0, KM=A15_PROD_21, KM8=A88_PROD_21, VN=A90_PROD_21, ATD=A43_PROD_0, F21=F21_PROD_21, F21DP=F21_PROD_21

> 🔑 **Translator design implication:** For MARS aliases, emit @[]@TableName (NOT {NODE_MAP[site]}.TableName). DataSyncX handles the substitution. Any hard-coded NODE_MAP in our translator is redundant for MARS.

3.4 MARS vs OASYS Discrimination Heuristic

When VG2 alias info has DATABASETYPE: Oracle, look for these OASYS hints:

- SINGLESCHEMA: "N/A" (MARS has Default)
- PROMPT-TEXT mentions "OASys"

- LOGICALVIEW starts with P_ (OASYS) vs F_ (MARS)
- Reference SQL uses /NODE=KM.OASYS

Best heuristic: LOGICALVIEW prefix (F_=MARS, P_=OASYS) backed by SINGLESCHEMA="N/A".

3.5 Processors (from Processors — DataSyncX latest documentation.pdf) [Processors...umentation | PDF]

- Extend datasyncx.core.base_component.Processor, implement process(self, df, *args, **kwargs) -> DataFrame.
- Task.processor accepts a single processor or a **list** (run in order, output piped).
- Built-in examples: RenameColumnsProcessor, DropColumnsProcessor.
- Best practice: small, composable, side-effect-free.

> 🔑 **Translator design implication:** Each VG2 transformation type (filter, computed-column, summary, pivot) is a candidate for its own processor class.

3.6 Task & TaskExecutor (the orchestration unit)

Minimal pattern:

```
task = Task(reader=..., writer=..., processor=[...], receiver="user@intel.com")
TaskExecutor().TaskRun(task=task)
```

3.7 TaskRangeBuilder & QueryBuilder

From the playbook DataSyncX ETL Agent Playbook — DataSyncX latest documentation.pdf: [DataSyncX...umentation | PDF]

- **TaskRangeBuilder** supports relative time (current_time - 12h, current_date - 7d) and absolute start/end, split by freq (1d, 2h, 30m).
- If QueryBuilder(..., timezone=True), relative current_time is UTC-first and SQL gets a TZ offset placeholder.

> 🔑 **Translator design implication:** VG2 For-Loop with date-range macros → TaskRangeBuilder with freq=<<<spf-step-i>>> and incremental relative expressions.

3.8 Three Build Paths (from FAQ + Copilot Agent doc) [FAQ — Data...umentation | PDF], [DataSyncX...umentation | PDF]

Path	When	Our use?
Free-style scripting	Full custom control	**Yes — our generated scripts will be free-style** so we can map the VG2 utility tree faithfully

Direct demo mode	Copy a Source_to_Target.py template	Useful as a per-output-type starting skeleton
Demo factory	YAML/JSON-driven generation	Possible v2 — if we emit a YAML spec instead of raw Python

3.9 MCP Pilot (from MCP Usage (Pilot) — DataSyncX latest documentation.pdf) [MCP Usage...umentation | PDF]

Read-only Copilot tools (mcp_health, mcp_list_capabilities, mcp_sqlserver_read_probe, mcp_mars_read_probe) that wrap the same readers. Useful as a **validation harness** for our translator — we can dry-run translated SQL through mcp_mars_read_probe before committing. **Not** part of the runtime translation path.

🔗 Section 4: The Translation Mapping (key cross-references for design)

VG2 Concept	DataSyncX Target	Notes
<ViewName> a0 (MARS, F_* prefix)	MarsReader().read(site, sql) with @[]@Table placeholders	Don't hardcode schema
<ViewName> a1 (OASYS, P_* prefix, SINGLESCHEMA=N/A)	OracleReader(database='OASYS').read(site, sql)	Explicit schema in SQL
SPF logical view (e.g., F_Lot_History_v3)	Hand-maintained **view-expansion YAML**	Must include physical tables, intra-view JOIN keys, implicit WHERE, bridge sub-aliases
Column header rename	RenameColumnsProcessor or DataFrame .rename()	
Filter grid	SQL WHERE clause for in-DB filters; processor for post-fetch refinements	In Temp → chunked SQL or pd.read_sql(... chunksize=...)
Computed column (scalar/CASE)	Custom processor using pandas/numpy	Map spf_FN$* → equivalent function
Computed column (analytical OVER)	Pandas groupby().transform() / .rank() / .shift()	
Statistic + GROUP BY	groupby().agg({...})	
Pivot (Row/Header/Value)	pd.pivot_table(...)	
For-Loop with date macros	TaskRangeBuilder (time mode)	

Site-Loop	Outer Python for site in sites: + per-site Task	
If-Then-Else	Native Python if/elif/else	
Smart-Append	DataSyncX merge writer with delete-by-date + update-by-key semantics	
Email, SharePoint	Task(receiver=...) for failures; custom notification for success	
Output File (CSV/TAB)	FileWriter (CSV)	

🏗 Section 5: Provisional Architecture Notes (to refine after I see real VG2 files)

The translation system likely needs **at least 5 cooperating components**, suggesting a **Pipeline + Visitor + Strategy** combination:

1. **VG2 Parser** — load + validate the XML/INI-like VG2 tree into an in-memory AST (one node class per VG2 element type).
2. **Symbol Resolver** — resolve global vars, env vars, command-line vars, system <<<spf-…>>> macros into a context dict.
3. **View Expander** — for each logical view alias, look up the view-expansion registry (YAML) and inject physical tables, joins, implicit WHEREs, bridge aliases. *This is the hand-maintained cornerstone.*
4. **Strategy-Per-Element Code Emitter** — Visitor pattern: each VG2 node type has an emitter that produces a Python AST fragment (preferably via ast module so we can ast.unparse). Strategies for: Reader selection (MARS vs Oracle), Processor chain construction, Utility-to-Python statement mapping, TaskRangeBuilder synthesis.
5. **Script Assembler + Validator** — stitch fragments into a runnable .py file, optionally run mcp_*_read_probe dry-runs against the emitted SQL.

Likely **design patterns** at play:

- **Pipeline** (parse → resolve → expand → emit → assemble)
- **Visitor** (per VG2 node type)
- **Strategy** (per reader / per utility kind)
- **Registry** (view-expansion YAML)
- **Builder** (the emitted Python script)
- **Adapter** (spf_FN$* → Python equivalents)

❓ Section 6: Open Questions I'll Need the VG2 Samples to Answer

1. **VG2 file format**: Is it XML, INI, or a custom serialization? The slides hint at a tree but don't show raw syntax — the actual files will clarify the parser strategy.

2. **Sub-alias chain syntax**: a0->p.prodgroup3 style — how nested can these get, and how are f0/p/v1/v2/v3 declared inside an alias block?
3. **The view-expansion registry scope**: How many distinct logical views does the team actually use? (This sizes the manual-effort cost.)
4. **Statistic + Level interaction**: When statistics live at higher Level values, do they apply to view-local results or all-sources results? (Matters for whether we push aggregation into SQL or do it in pandas.)
5. **Utility nesting depth**: Real queries often have If-Then inside For-Loop inside Site-Loop — what's the typical depth and which <<<spf-loop-ctr-i>>> patterns are used?
6. **Smart-Append config in real use**: What delete columns + criteria are typical? (Drives the writer-mode mapping.)

☑ Self-Check: Grounding Confidence

Area	Confidence	Source
DataSyncX reader API & schema substitution	**High**	datasyncx_reader_fact.txt (verified from venv source) [datasyncx_...eader_fact | Txt]
VG2 utility catalog	**High**	Adding Utilities - SQLPathFinder - Intel Enterprise Wiki.pdf (71-min read, comprehensive) [Adding Uti...prise Wiki | PDF]
Column/Filter/Pivot/Summary semantics	**High**	All four self-paced PPSM decks [Introducti...d_Pivoting | PowerPoint], [Introducti...ed_Summary | PowerPoint], [Introducti..._Columns_2 | PowerPoint], [Introducti..._Filtering | PowerPoint]
Global/system variables	**High**	`Global Variables - SQLPathFinder - Intel Enterprise Wiki.pdf [Global Var...prise Wiki | PDF]
DataSyncX architecture & Task/Processor model	**High**	Concepts + Processors PDFs [Concepts —...umentation | PDF], [Processors...umentation | PDF]
Exact VG2 on-disk syntax	**Low — awaiting real samples**	Not specified in docs
Per-team SPF view-expansion details	**Low — needs registry build**	Flagged in reader_fact.txt as a known gap [datasyncx_...eader_fact | Txt]